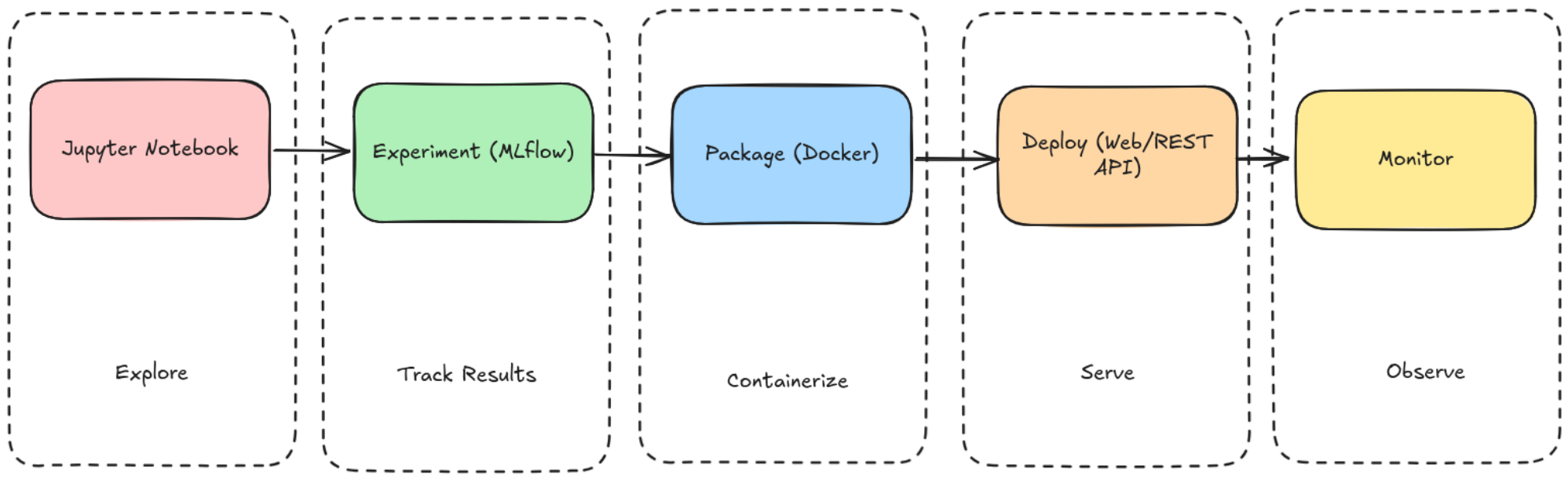
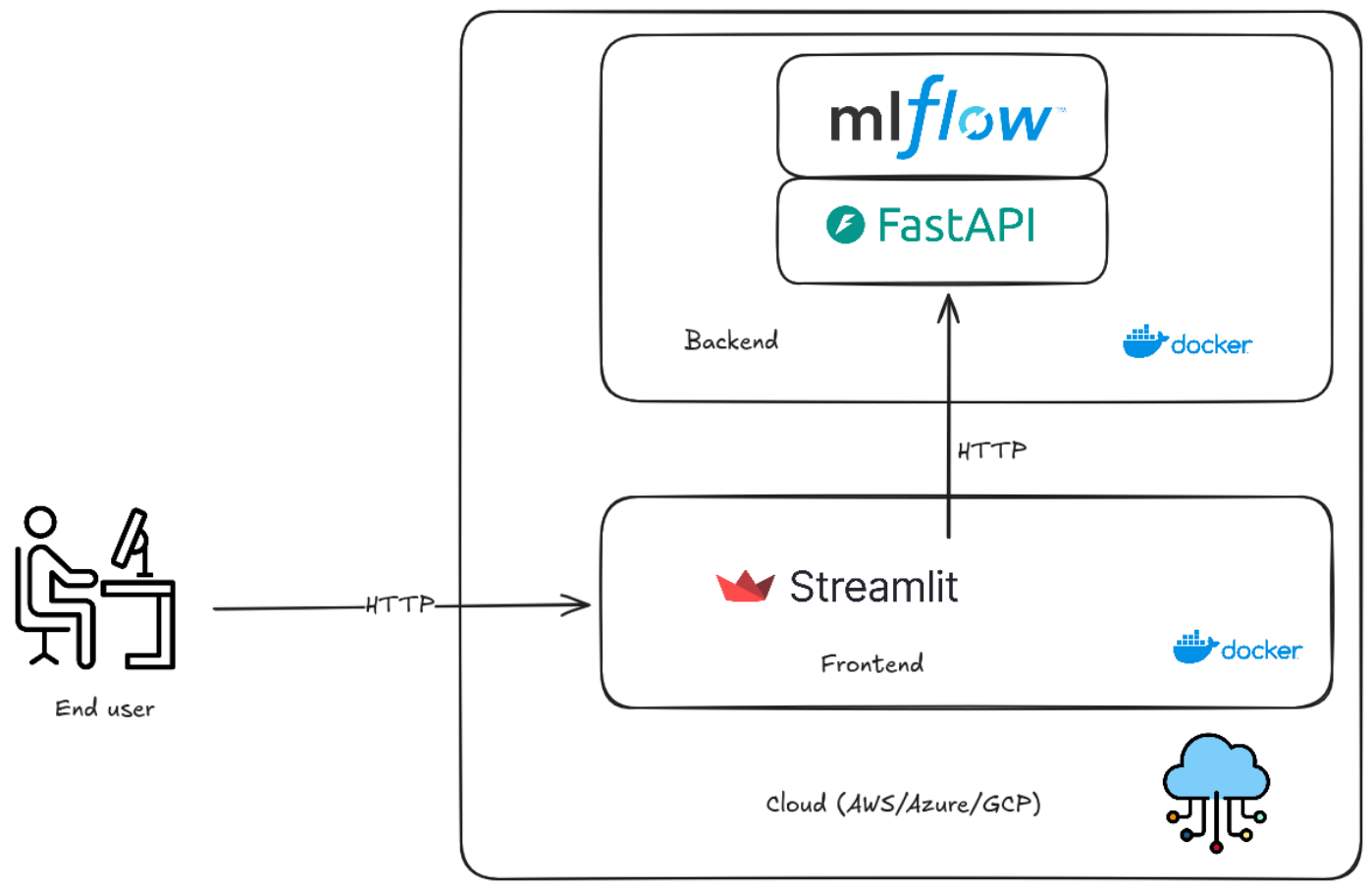
Serving & Monitoring

From Notebook to Production

The Journey:



The Production ML Stack



Part 1: Model Serving Strategies

What is Model Serving?
Making your trained model available to make predictions on new data

Approaches:

1. **REST API** - Real-time predictions via HTTP endpoints
2. **Batch Processing** - Predictions on large datasets periodically
3. **Streaming** - Real-time predictions on data streams

Choosing the Right Approach: Depends on latency, throughput, and use case

Deployment Strategy 1: REST APIs

Use Case: Real-time predictions with low-to-medium latency

Characteristics:

- Synchronous request/response
- Latency: milliseconds to seconds
- Examples: Fraud detection, recommendation systems, chatbots

Deployment Strategy 1: REST APIs

Tools:

- FastAPI (Python)
- Flask (Python)
- MLflow Models Serving
- Cloud services (SageMaker, Azure ML, Vertex AI)

Building REST API with FastAPI

Why FastAPI?

- Fast performance (on par with Node.js)
- Automatic API documentation (Swagger/OpenAPI)
- Type validation with Pydantic
- Async support
- Easy to test and deploy

Basic structure: TODO - Add code example

Deployment Strategy 2: Batch Predictions

Use Case: Predictions on large datasets, not time-sensitive

Characteristics:

- Asynchronous processing
- Latency: minutes to hours
- Examples: Email campaigns, monthly reports, ETL pipelines

Deployment Strategy 2: Batch Predictions

Implementation:

- Scheduled jobs (cron, Airflow)
- Spark/Dask for large-scale processing
- Cloud batch services (AWS Batch, GCP Dataflow)

Deployment Strategy 3: Streaming Predictions

Use Case: Real-time predictions on continuous data streams

Characteristics:

- Continuous processing
- Low latency (sub-second)
- Examples: IoT sensors, real-time analytics, financial trading

Deployment Strategy 3: Streaming Predictions

Tools:

- Apache Kafka + Kafka Streams
- Apache Flink
- AWS Kinesis
- Google Cloud Dataflow

Comparison: Serving Strategies

Strategy	Latency	Throughput	Use Case	Complexity
REST API	Low (ms)	Medium	Real-time individual predictions	Low
Batch	High (hrs)	High	Periodic bulk predictions	Low
Streaming	Very Low (ms)	Very High	Continuous real-time	High

Introduction to Streamlit

What is Streamlit?

- Python framework for building web apps
- No HTML/CSS/JavaScript required
- Rapid prototyping and deployment
- Built-in widgets and visualizations

Introduction to Streamlit

Perfect for:

- Model demos and prototypes
- Internal dashboards
- Quick MVPs

Streamlit: First App

TODO - Add basic Streamlit example

Streamlit: ML Model Demo

TODO - Add Streamlit app with model predictions

Streamlit Deployment

Deployment Options:

1. **Streamlit Cloud** (free tier available)
2. **Docker + Cloud** (AWS, GCP, Azure)
3. **Heroku**
4. **Self-hosted**

Part 3: Scaling Production ML

Scaling Challenges

As your ML system grows:

- More users → Higher traffic
- Larger models → More memory/compute
- More data → Slower processing
- More features → Complex pipelines

Scaling Challenges

Solutions:

- Horizontal scaling (more instances)
- Vertical scaling (bigger instances)
- Model optimization (quantization, pruning)
- Caching and load balancing

Horizontal Scaling with Load Balancers

Components:

- Load balancer (nginx, AWS ALB)
- Multiple model server instances
- Auto-scaling based on metrics

Containerization for Scaling

Using Docker + Kubernetes:

Benefits:

- Easy scaling (kubectl scale)
- Self-healing
- Rolling updates
- Resource management

Why Monitor ML Systems?

Things to Track in Production:

- System performance (latency, throughput)
- Error rates and types
- Resource usage (CPU, memory)
- Request patterns
- API availability

Why Monitor ML Systems?

Without Monitoring:

- No visibility into issues
- Slow incident response
- Poor user experience

Basic Monitoring Metrics

1. **Performance Metrics**
   - Response time (latency)
   - Requests per second (throughput)
   - Success/error rate
2. **System Health**
   - CPU usage
   - Memory usage
   - Disk space

Basic Monitoring Metrics

3. **API Metrics**
    - Endpoint usage
    - Status codes (200, 400, 500)
    - Request size/response size

Simple Monitoring with Logging

Built-in Python Logging:

```
import logging

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)

logger = logging.getLogger(__name__)
```

Simple Monitoring with Logging

Built-in Python Logging:

```
# In your API
@app.post("/predict")
async def predict(data: Input):
    logger.info(f"Received prediction request")
    prediction = model.predict(data)
    logger.info(f"Prediction: {prediction}")
    return {"prediction": prediction}
```

Production Logging Best Practices

What to Log:

1. **Input data** (for debugging)
2. **Predictions** (for auditing)
3. **Prediction confidence** (for quality monitoring)
4. **Latency** (for performance tracking)
5. **Errors and exceptions**
6. **Feature values** (for drift detection)

Production Logging Best Practices

How to Log:

- Structured logging (JSON)
- Centralized logging (ELK, CloudWatch)
- Different log levels (DEBUG, INFO, WARNING, ERROR)

Setting Up Alerts

When to Alert:

- Model accuracy drops below threshold
- Data drift detected
- High error rate (>1%)
- Latency exceeds SLA
- System resource exhaustion

Setting Up Alerts

Alert Channels:

- Email
- Slack/Teams
- PagerDuty
- SMS for critical issues

Alert Configuration Example

Using FastAPI with Status Tracking:

```
from fastapi import FastAPI, HTTPException
import logging

app = FastAPI()
logger = logging.getLogger(__name__)

# Track error counts
error_count = 0
MAX_ERRORS = 10
```

Alert Configuration Example

```python
@app.post("/predict")
async def predict(data: Input):
    global error_count
    try:
        prediction = model.predict(data.features)
        error_count = 0  # Reset on success
        return {"prediction": prediction}
    except Exception as e:
        error_count += 1
        logger.error(f"Prediction failed: {e}")
        if error_count >= MAX_ERRORS:
            logger.critical("Too many errors! Alert needed!")
            # Send alert (email, Slack, etc.)

        raise HTTPException(status_code=500, detail=str(e))
```

Why FastAPI for ML?

Advantages:

- ✅ **Fast**: High performance (async support)
- ✅ **Type validation**: Pydantic models
- ✅ **Auto docs**: Swagger UI out of the box
- ✅ **Easy testing**: Built-in test client
- ✅ **Production ready**: Used by Netflix, Uber, Microsoft

Why FastAPI for ML?

Perfect for ML APIs:

- Input validation critical for models
- Automatic API documentation
- Easy to add authentication
- Scalable architecture

FastAPI: Basic Structure

```python
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

# Load model
model = joblib.load("model.pkl")

# Define FastAPI app
app = FastAPI(title="ML Prediction API")

# Define input schema
class PredictionInput(BaseModel):
    feature1: float
    feature2: float
    feature3: float

# Define output schema
class PredictionOutput(BaseModel):
    prediction: float
    confidence: float

@app.post("/predict", response_model=PredictionOutput)
def predict(input_data: PredictionInput):
    # Convert to format model expects
    features = [[input_data.feature1, input_data.feature2, input_data.feature3]]

    # Make prediction
    prediction = model.predict(features)[0]
    confidence = model.predict_proba(features).max()

    return {
        "prediction": float(prediction),
        "confidence": float(confidence)
    }
```

FastAPI: Running the Server

Start the server:

```
# Install FastAPI and uvicorn
pip install fastapi uvicorn

# Run the server
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

FastAPI: Running the Server

Automatic documentation:

- Interactive docs: `http://localhost:8000/docs`
- Alternative docs: `http://localhost:8000/redoc`

Test with curl:

```
curl -X POST "http://localhost:8000/predict" \
  -H "Content-Type: application/json" \
  -d '{"feature1": 1.0, "feature2": 2.0, "feature3": 3.0}'
```

FastAPI: Advanced Features

- Multiple Endpoints
- Built in authentication
- Built in testing
- Easy HTTP Error Handling

Streamlit: Why Use It?

Perfect for:

- Quick ML demos
- Internal dashboards
- Data exploration tools
- Prototypes and MVPs

Streamlit: Why Use It?

Advantages:

- ✅ Pure Python (no HTML/CSS/JS)
- ✅ Reactive updates
- ✅ Built-in widgets
- ✅ Easy deployment
- ✅ Beautiful by default

Streamlit: Why Use It?

When NOT to use:

- Complex multi-page apps with custom routing
- High-performance production APIs
- Apps requiring fine-grained control

Streamlit: Basic App Structure

Hello World:

```
import streamlit as st

# Title
st.title("My First Streamlit App")

# Text
st.write("Hello, Streamlit!")

# Input widget
name = st.text_input("What's your name?")

if name:
    st.write(f"Hello, {name}!")
```

Streamlit: Basic App Structure

Run it:

```
pip install streamlit
streamlit run app.py
```

Opens at `http://localhost:8501`

Streamlit: ML Prediction App

```
import streamlit as st
import joblib
import pandas as pd

# Load model
@st.cache_resource
def load_model():
    return joblib.load("model.pkl")

model = load_model()

# Title and description
st.title("ML Prediction App")
st.write("Enter feature values to get a prediction")
```

Streamlit: ML Prediction App

```
# Create input form
with st.form("prediction_form"):
    st.subheader("Input Features")

    feature1 = st.number_input("Feature 1", min_value=0.0, max_value=10.0, value=5.0)
    feature2 = st.number_input("Feature 2", min_value=0.0, max_value=10.0, value=5.0)
    feature3 = st.number_input("Feature 3", min_value=0.0, max_value=10.0, value=5.0)

    submitted = st.form_submit_button("Predict")
```

Streamlit: ML Prediction App

```
# Make prediction
if submitted:
    features = [[feature1, feature2, feature3]]
    prediction = model.predict(features)[0]

    st.success(f"Prediction: {prediction:.2f}")

    # Show prediction probability if available
    if hasattr(model, 'predict_proba'):
        proba = model.predict_proba(features)[0]
        st.bar_chart(pd.DataFrame({'Probability': proba}, index=model.classes_))
```

Streamlit: Advanced Widgets

Rich Input Options:

```
import streamlit as st

# Sliders
age = st.slider("Age", 0, 100, 25)

# Select box
model_type = st.selectbox("Model", ["RandomForest", "LogisticRegression", "XGBoost"])

# Multi-select
features = st.multiselect("Select features", ["age", "income", "education"])
```

Streamlit: Advanced Widgets

```
# File uploader
uploaded_file = st.file_uploader("Upload CSV", type=['csv'])
if uploaded_file:
    df = pd.read_csv(uploaded_file)
    st.dataframe(df)

# Sidebar
with st.sidebar:
    st.header("Settings")
    threshold = st.slider("Confidence threshold", 0.0, 1.0, 0.5)

# Columns
col1, col2 = st.columns(2)
with col1:
    st.metric("Accuracy", "95%")
with col2:
    st.metric("Predictions", "1,234")
```

Streamlit: Data Visualization

Built-in Charts:

```
import streamlit as st
import pandas as pd
import numpy as np

# Line chart
chart_data = pd.DataFrame(
    np.random.randn(20, 3),
    columns=['a', 'b', 'c']
)
st.line_chart(chart_data)

# Area chart
st.area_chart(chart_data)
```

Streamlit: Data Visualization

Built-in Charts:

```
# Bar chart
st.bar_chart(chart_data)

# Map (for geospatial data)
map_data = pd.DataFrame(
    np.random.randn(100, 2) / [50, 50] + [37.76, -122.4],
    columns=['lat', 'lon']
)
st.map(map_data)
```

Streamlit: Integrations

Matplotlib:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [1, 4, 2, 3])
ax.set_title("Sample Plot")
st.pyplot(fig)
```

Streamlit: Integrations

Plotly (Interactive):

```
import plotly.express as px

df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species")
st.plotly_chart(fig)
```